ANDROID APP DEVELOPMENT USING GEN AI INTERNSHIP PROGRAM

Layout Plan –
Happy Birthday Greeting Screen

A Structured Layout & Resource Planning Document

Student Name:	Guru K.R.
Course:	Android Internship Program
Task:	Task 3 — Planning a Simple Layout with Text + Image

1. Screen Purpose

The Happy Birthday Greeting Screen is a simple, single-page Android UI designed to display a warm and visually engaging birthday message to a specified recipient. Its primary purpose is to present a celebratory image, a bold greeting title, the recipient's name as a subtitle, and a personalised message, concluding with an interactive button that allows the user to send or share the greeting. This screen serves as a practical introduction to composing multi-element vertical layouts in Jetpack Compose.

2. Layout Wireframe

The wireframe below represents the intended visual structure of the Happy Birthday Greeting Screen prior to implementation. It communicates the relative positioning, sizing, and hierarchy of all UI elements, serving as the primary reference for the composable structure plan outlined in the subsequent section.

Code

```
--------------------------------
|        🎂 Image              |
--------------------------------

        Happy Birthday!

        Dear Rahul,
```

Wireframe Element Breakdown

The screen is structured as a single vertical stack from top to bottom, comprising the following elements in order:

- **Celebration Image (Top):** A birthday cake or celebratory illustration occupies the upper portion of the screen. It spans the full available width with a fixed height, providing immediate visual context for the screen's purpose.
- **Title Text:** A large, bold "Happy Birthday!" heading is centred immediately below the image. Its prominent size ensures it is the first text element the user reads.
- **Subtitle Text (Recipient Name):** A smaller text element displaying the recipient's name appears below the title, maintaining centre alignment and providing personalisation.
- **Greeting Paragraph:** A short, warm body text message is placed beneath the subtitle, providing additional context or a personalised birthday note in a readable font size.
- **Action Button (Bottom):** A centred "Send Wishes" button is anchored near the bottom of the screen, providing a clear call-to-action for the user.

3. Android Studio Implementation Screenshots

The following screenshots were captured during the implementation and testing phase of the Happy Birthday Greeting Screen within Android Studio. They document the Kotlin source code structure and the rendered output on the Pixel 6 API 36 emulator, confirming that the planned layout was successfully realised.

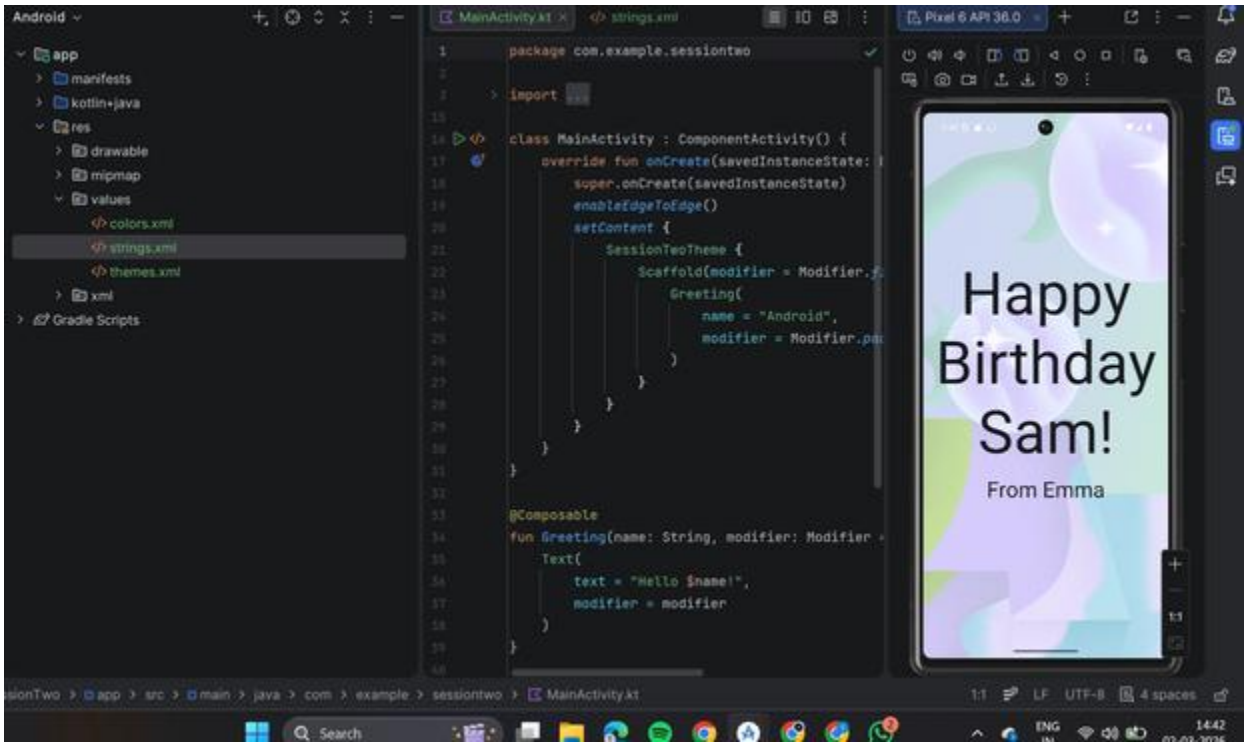

Figure 1 — Android Studio IDE showing MainActivity.kt with the Greeting composable and the Happy Birthday screen rendered on the Pixel 6 emulator (API 36)

Figure 2 (left) — Scaffold with Modifier.fillMaxSize() and innerPadding applied to the Greeting composable | Figure 3 (right) — Greeting composable function and @Preview annotation with GreetingPreview()

Figure 4 — Pixel 6 emulator (API 36) showing the rendered Happy Birthday greeting: "Happy Birthday Sam! / From Emma"

4. Composable Structure Plan

The following composables have been identified as the most appropriate building blocks for constructing the Happy Birthday Greeting Screen. Each composable has been selected based on its functional role within the layout hierarchy and its suitability for the specific UI element it represents.

Column

The root layout composable for this screen. Column arranges all child elements vertically from top to bottom, reflecting the natural reading flow of the greeting screen. It will be configured with centre horizontal alignment and will occupy the full screen size using Modifier.fillMaxSize(). A uniform padding of 16dp will be applied on all sides to prevent content from touching the screen edges.

Image

Used to display the birthday cake or celebration illustration at the top of the screen. The Image composable will load the birthday_cake.png resource from the drawable-nodpi folder. A content scale of ContentScale.Fit will be applied to ensure the image is displayed proportionally within its designated height, without distortion.

Text (Title)

A large, bold Text composable will display the primary greeting message "Happy Birthday!". It will use a font size of 28sp, centre text alignment, and the primaryColor token from the application's Material theme to maintain visual consistency.

Text (Subtitle)

A secondary Text composable displaying the recipient's name will be placed immediately below the title. It will use a font size of 20sp with a slightly lighter font weight to establish visual hierarchy between the title and the name.

Text (Greeting Message)

A body Text composable presenting a short personalised greeting paragraph will appear below the subtitle. It will use a font size of 16sp and the onBackgroundColor token for readability against the screen background.

Button

A single Button composable labelled "Send Wishes" will be positioned near the bottom of the Column. It will use a font size of 14sp for its label text and will be styled using the primary button colour from the Material theme. A top padding of 16dp will separate it from the greeting text above.

Modifier.padding

The Modifier.padding composable modifier will be applied throughout the layout to introduce spacing between elements. Key values include 16dp between the image and the title, 8dp between the title and the subtitle, 8dp between the subtitle and the greeting message, and 16dp between the message and the action button.

5. Spacing & Layout Notes

Consistent spacing is critical to achieving a clean, readable, and visually balanced layout. The following spacing values have been determined based on Material Design guidelines and the visual weight of each element within the screen.

Element Pair	Spacing Value	Alignment
Image → Title Text	16dp	Centre (horizontal)
Title Text → Subtitle Text	8dp	Centre (horizontal)
Subtitle Text → Greeting Message	8dp	Centre (horizontal)
Greeting Message → Send Wishes Button	16dp	Centre (horizontal)
Screen edge → Column content	16dp (all sides)	N/A

All text elements within the Column will be horizontally centred using textAlign = TextAlign.Center. The Column itself will be configured with horizontalAlignment = Alignment.CenterHorizontally to ensure all children inherit central positioning without requiring individual alignment modifiers on each composable.

6. Resource Plan

6.1 Image Resource

Property	Value
File Name	birthday_cake.png
Folder	res / drawable-nodpi
Format	PNG (lossless, supports transparency)
Approx. Size	Medium resolution — optimised for display (recommended: 400 × 400 px)
Content Scale	ContentScale.Fit — preserves aspect ratio within bounds

Description	A birthday cake or celebration-themed illustration used as the hero image at the top of the greeting screen

6.2 Typography Resource Plan

Text Element	Font Size	Font Weight	Colour Token
Title (Happy Birthday!)	28sp	Bold	primaryColor
Subtitle (Name)	20sp	Medium	onSurfaceColor
Body Message	16sp	Normal	onBackgroundColor
Button Label	14sp	Medium	onPrimaryColor (White)

6.3 Colour Token Reference

All colour values will be sourced from the application's Material Design colour scheme rather than hard-coded hexadecimal values. This ensures consistency across light and dark theme variants and simplifies future theming changes. The key tokens used in this screen are as follows:

- **primaryColor:** Applied to the title text and button background. Represents the brand's main accent colour.
- **backgroundColor:** The screen's background fill, drawn from the Material theme's surface colour token.
- **onPrimaryColor:** Applied to the button label text to ensure legibility over the primary-coloured button background.
- **onSurfaceColor:** Used for the subtitle text to maintain readable contrast against the background.
- **onBackgroundColor:** Applied to the body greeting text for standard body-copy legibility.

7. Reflection

Planning the composable structure and resource specifications prior to writing any implementation code proved to be an invaluable exercise in structured thinking. By identifying the appropriate composables — Column, Image, Text, and Button — and defining their roles within the layout hierarchy in advance, the development process became significantly more focused and efficient. There was no need to refactor the overall structure mid-implementation, as the vertical stacking order and element responsibilities had already been resolved at the planning stage.

Defining spacing values such as 16dp and 8dp in the planning document meant that visual consistency could be maintained from the very first build, rather than discovered and corrected iteratively through trial and error. Similarly, pre-selecting font sizes and colour tokens ensured that the typography hierarchy — from the 28sp bold title down to the 14sp button label — communicated the intended visual weight relationships without ambiguity.

The practice of planning resources — specifically the image file name, folder location, resolution, and content scale — prevented common pitfalls such as missing drawable references and oversized image assets that can cause slow rendering or build errors. Overall, this planning phase demonstrated that a small investment of time in structured documentation before coding yields measurable savings in debugging time, rework effort, and final layout quality, a discipline that will remain central to all subsequent development tasks throughout this internship programme.